Introduction à la Programmation et l'Algorithmique

DINO LOPEZ PACHECO

DINO.LOPEZ@UNIV-COTEDAZUR.FR

La Récursivité

Récursivité *in a nutshell*

- Classiquement, une fonction appelle une autre fonction;
- Une fonction peut s'appeler elle-même :
  - On parle d'une fonction récursive.
- La récursivité n'est pas appropriée à tous les problèmes :
  - Les nombreuses appels peuvent faire déborder la pile
  - Privilégier les fonctions itératives.
- Pour créer une fonction récursive, nous devons :
  - Définir le cas basique;
  - Définir les étapes récursives.

Récursive vs Itérative

L'analyse du problème peut conduire à une solution itérative ou récursive. Exemple avec la fonction factorielle

$$n! = n \times (n-1) \times \cdots \times 2 \times 1 = \prod_{k=1}^{n} k$$

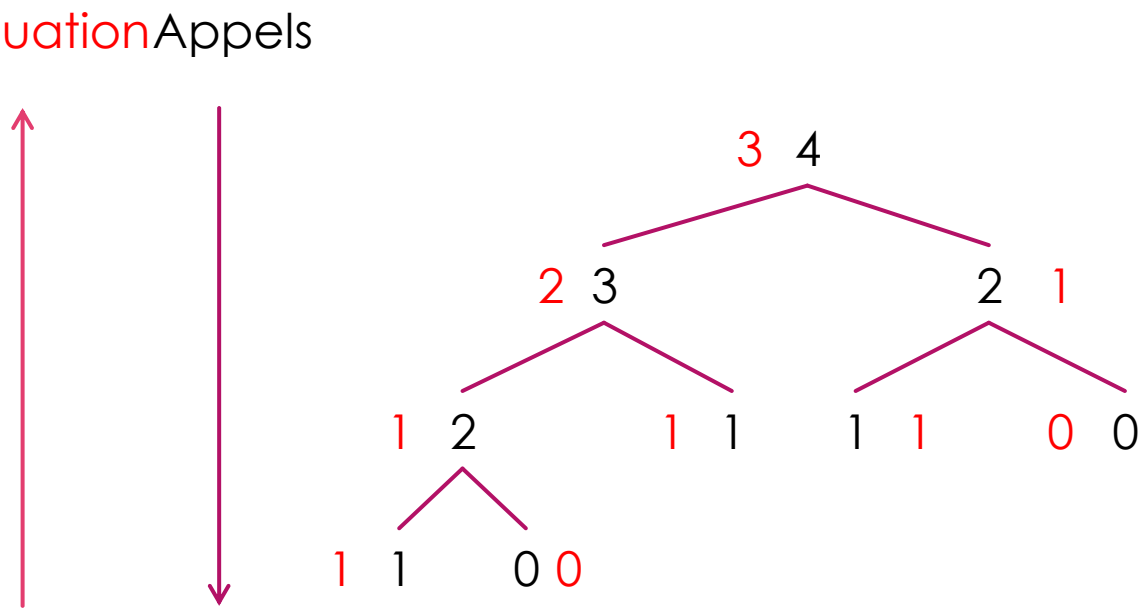
```
fonction factorielle (n) {
  fact = 1
  pour i de 1 à n, faire {
    fact = fact * i
  }
  retour fact
}
```

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \times (n-1)! & \text{if } n > 0 \end{cases}$$

```
fonction factorielle (n) {
  si (n == 0) {
    retour 1
  } sinon {
    retour n * factorielle(n-1)
  }
}
```

= Les danger de la récursivité = L'exemple avec Fibonacci

```
fonction fibonacci (n) {
  si (n == 0) retour 0
  si (n == 1) retour 1
  retour fibonacci(n-1) + fibonacci(n-2)
}
```



Bel algorithme… mais très loin de l'optimal
Penser à la complexité

Entrée des programmes : Scanner

java.util.Scanner

« Alimentation » d'un programme

- Avec des arguments au moment du lancement du programme
  - main(String[] args)
  - Solution à privilégier car facile à automatiser les lancement des programmes à partir d'un script shell
  - Interaction privilégié par les commandes systèmes
- Interaction par l'entrée standard
  - Travail en mode « interactif »
  - Difficile à automatiser (redirection du STDIN)
  - Pratique pour les tests unitaires
- Interaction par de fichiers
  - Très pratique lorsque le programme doit traiter une grande quantité de données

Scanner

- « Scanner » permet d'ouvrir un dispositif d'entrée (fichier ou autre dispositif)
  - Très souvent employé pour « ouvrir » STDIN (System.in)
- Offre plusieurs méthodes pour savoir si de données attendent d'être lues
  - hasNext(), hasNextInt(), hasNextDouble(), hasNextLine(), etc.
- Et des méthodes pour lire le dispositif d'entrée
  - next(), nextLine(), nextInt(), nextDouble(), etc.
- Une ressource ouverte doit être fermée
  - *Best practice*
  - Sauf pour STDIN
- Fermer Scanner == fermer le dispositif de lecture
  - Une fois STDIN fermé, ce n'est plus possible de l'ouvrir

Exemple

```
public class TestScanner {
  Scanner s;

  public TestScanner() {
    s = new Scanner(System.in);
  }

  public String getKeyboardInput() {
    String input="";
    while (s.hasNext()) {
      input = s.nextLine();
      if (input.equals("exit"))
        break;
    }
    return input;
  }

   public static void main(String[] args) {
      TestScanner ts = new TestScanner();
      System.out.println(ts.getKeyboardInput());
      System.out.println("new");
      System.out.println(ts.getKeyboardInput());
    }
}
```